

GATE & COMPETITIVE EXAMS QUESTION BANK

50 Solved Questions on Functions, Arrays, and Recursion

Part 1: Functions & Scope Rules

GATE CS 2018

Q1. A function contains the code `static int count = 0; count++;` . If this function is called 3 times sequentially, what will be the final value of `count` ?

SOLUTION & LOGIC:

`static` variables are initialized only **once** (at compile time) and retain their values across multiple function calls.

- Call 1: `count` increments from 0 to 1.
- Call 2: `count` increments from 1 to 2.
- Call 3: `count` increments from 2 to 3.

Answer: 3

GATE CS 2016

Q2. Consider the function `void swap(int x, int y) { int temp = x; x = y; y = temp; }` . If called using `swap(a, b)` , will the values of `a` and `b` change in the main function?

SOLUTION & LOGIC:

This is an implementation of **Call by Value**. The parameters `x` and `y` are local copies of the actual arguments. Modifications inside the function do not affect the original variables `a` and `b` .

Answer: No change in original values.

GATE CS 2015

Q3. Given: `int x = 10; void f() { print(x); } void g() { int x = 20; f(); } .` What will be the outputs if the language uses Static Scoping vs Dynamic Scoping?

SOLUTION & LOGIC:

- **Static Scoping:** The compiler looks at the lexical structure. `f()` looks for `x` in its global definition environment. Output = 10.
- **Dynamic Scoping:** The runtime system looks at the calling stack sequence. `f()` is called by `g()`, which has a local `x = 20`. Output = 20.

Answer: Static = 10, Dynamic = 20.

ISRO CS 2017

Q4. Predict the output of the following block:

```
{ int x = 5; { int x = 10; } printf("%d", x); } .
```

SOLUTION & LOGIC:

C supports block scopes. The inner variable `x = 10` becomes inaccessible once execution exits its enclosing curly braces `{}`. The `printf` statement references the outer `x`.

Answer: 5

NIELIT SCIENTIST B 2019

Q5. A program defines a global variable `int a = 5;` and inside `main()` defines a local variable `int a = 10;`. What is printed by `printf("%d", a);` inside main?

SOLUTION & LOGIC:

When a local variable shares the same identifier as a global variable, the local variable **shadows/hides** the global variable within its local scope.

Answer: 10

GATE CS 2011

Q6. What structures are safely pushed onto the stack inside an Activation Record (Stack Frame) during a function call?

SOLUTION & LOGIC:

An Activation Record holds execution context, including: Local variables, Return address (to resume execution in the caller), Formal parameters, and Saved frame pointers.

Answer: Local variables, Parameters, and Return Address.

UGC NET CS 2020

Q7. Match parameter passing mechanisms: Call by value, Call by reference, Call by name with their traits.

SOLUTION & LOGIC:

- **Call by value:** Creates an isolated duplicate copy.
- **Call by reference:** Passes the address/alias directly to the function.
- **Call by name:** Lazy evaluation, works via textual substitution (similar to Algol macros).

GATE CS 2008

Q8. What does the declaration `int (*ptr)(float);` represent?

SOLUTION & LOGIC:

The syntax binds the pointer symbol `*ptr` inside parentheses, creating a **Function Pointer** that targets a function accepting a `float` parameter and returning an `int`.

Answer: Pointer to a function taking float and returning int.

ISRO CS 2020

Q9. Given the macro definition `#define CUBE(x) x*x*x` , evaluate the macro expansion outcome of `CUBE(1+2)` .

SOLUTION & LOGIC:

Macros perform naive text substitution before compilation.

`CUBE(1+2)` transforms to: `1+2*1+2*1+2` .

By standard mathematical operator precedence, multiplication runs first: `1 + (2*1) + (2*1) + 2 = 1 + 2 + 2 + 2 = 7` .

Answer: 7

COMPETITIVE EXAM STANDARD

Q10. What is a mechanism where a function accepts a function pointer to execute it later called?

SOLUTION & LOGIC:

Passing a reference of a functional block as an argument to another block is formally known as a **Callback Mechanism**.

Answer: Function Callback.

GATE CS 2004

Q11. Are `inline` functions always expanded inline by compilers when they contain recursive logic?

SOLUTION & LOGIC:

The `inline` keyword is a recommendation to the compiler. Since recursive depth is usually dynamic, a compiler cannot determine code size expansions at compile-time and safely treats it as a standard function.

Answer: No, compilers ignore inline optimizations for recursive functions.

Q12. Differentiate between Formal Parameters and Actual Parameters.

SOLUTION & LOGIC:

- **Formal Parameters:** Variables declared inside the function's signature definition (e.g., `void fun(int x)`).
- **Actual Parameters:** Real variables or expressions evaluated and passed during runtime execution calls (e.g., `fun(y)`).

Part 2: Arrays & Pointer Arithmetic

Q13. A 2D array defined as $A[10][20]$ is stored using Row-Major Order. If the Base Address is 2000 and element size is 4 bytes, compute the target memory address of $A[5][10]$.

SOLUTION & LOGIC:

Using the Row-Major formula: $\text{Address} = \text{Base Address} + [i \times \text{Total Columns} + j] \times W$

$$\text{Address} = 2000 + [5 \times 20 + 10] \times 4$$

$$\text{Address} = 2000 + [100 + 10] \times 4 = 2000 + (110 \times 4) = 2000 + 440 = 2440$$

Answer: 2440

Q14. Write the formula to locate element $A[i][j][k]$ in a 3D array sizing $d_1 \times d_2 \times d_3$ arranged under Column-Major scheme.

SOLUTION & LOGIC:

In Column-Major configuration, the first index varies fastest, followed by subsequent dimensions.

$$\text{Formula: } \text{ext}\{\text{Address}\} = \text{ext}\{\text{Base Address}\} + ((k \times d_2 + j) \times d_1 + i) \times \text{ext}\{\text{Size}\}$$

GATE CS 2014

Q15. Given `int a[] = {10, 20, 30}; int *p = a;` , differentiate between expressions `*p++` and `(*p)++` .

SOLUTION & LOGIC:

- `*p++` : Post-increment runs on pointer. It evaluates the current element value `*p` (10), and then increments the pointer `p` to target the next sequential element address.
- `(*p)++` : The post-increment hits the dereferenced internal element data. The current data value (10) is loaded, and then the value inside that position increases to 11 while the pointer stays still.

ISRO CS 2018

Q16. Given `int arr[5];` , what is the core structural difference between evaluating `arr + 1` versus `&arr + 1` ?

SOLUTION & LOGIC:

Both start at the same numeric Base Address, but their underlying pointer types differ:

- `arr` is a pointer to the first single element. `arr + 1` steps forward by 1 element size (**1 times 4 = 4 bytes**).
- `&arr` is a pointer referencing the entire block array of 5 units. `&arr + 1` steps forward by the entire array span size (**5 times 4 = 20 bytes**).

NIELIT 2021

Q17. Predict the printed output: `char *ptr = "GATE2026"; ptr++; printf("%s", ptr);` .

SOLUTION & LOGIC:

The initialization maps `ptr` to character 'G'. Running `ptr++` shifts the pointer destination forward to point to 'A'. The `%s` string format directive streams data until a null character terminates it.

Answer: ATE2026

GATE CS 2015

Q18. Differentiate between declarations `int *ptr[5]` and `int (*ptr)[5]` .

SOLUTION & LOGIC:

- `int *ptr[5]` : An **Array of Pointers**. It declares an array holding 5 independent integer pointer locations.
- `int (*ptr)[5]` : A **Pointer to an Array**. It defines a single pointer addressing a dedicated block array layout containing 5 integers.

UGC NET 2019

Q19. What is the standard worst-case time complexity of standard matrix multiplication algorithm over two $N \times N$ dimensions?

SOLUTION & LOGIC:

The naive iterative algorithm involves 3 nested loops looping N times to multiply rows by columns.

Answer: $O(N^3)$

GATE CS 2006

Q20. In a Lower Triangular Matrix $A[n][n]$ using 1-based indexing, find the linear 1D index mapping for cell $A[i][j]$ in Row-Major arrangement.

SOLUTION & LOGIC:

Rows preceding row i hold $1 + 2 + \dots + (i-1) = \frac{i(i-1)}{2}$ elements. In row i , we count up to column j .

Formula: Index = $\frac{i(i-1)}{2} + j$

COMPETITIVE EXAM STANDARD

Q21. Explain the architectural setup of Jagged Arrays.

SOLUTION & LOGIC:

A jagged array is a multidimensional construct where rows are allocated dynamically with varying column lengths. It is implemented as an array of pointers to distinct memory blocks.

GATE CS 2017

Q22. Analyze the result of this array reversal logic: `for(i=0; i<n; i++) swap(&a[i], &a[n-i-1]);` .

SOLUTION & LOGIC:

The loop correctly flips elements up to the midpoint ($n/2$). However, continuing past the midpoint swaps the elements back to their original positions.

Answer: The array content remains unchanged.

ISRO CS 2015

Q23. What specific signal pointer does `malloc()` return if system heap memory is exhausted?

SOLUTION & LOGIC:

When a program requests memory that the operating system cannot allocate, `malloc()` fails safely and returns a `NULL` pointer.

Answer: NULL

GATE CS 2012

Q24. Evaluate `sizeof(arr)` inside `void f(int arr[10])` on a 64-bit platform architecture (pointer size = 8 bytes).

SOLUTION & LOGIC:

When passed to a function, an array decays into a pointer to its first element. Therefore, `sizeof(arr)` returns the size of the pointer rather than the array.

Answer: 8 bytes

UGC NET 2021

Q25. What execution behavior takes place when executing out-of-bounds statements like `int a[5]; a[10] = 20;` in C?

SOLUTION & LOGIC:

C does not perform runtime array bound-checking. Writing to an invalid index causes **Undefined Behavior**, which can corrupt data or trigger a Segmentation Fault.

Answer: Undefined Behavior.

Part 3: Pure Recursion & Tree Recursion

GATE CS 2016

Q26. Trace the value returned by calling `f(4)` :

```
int f(int n) {  
    if (n <= 1) return 1;  
    return n + f(n-1);  
}
```

SOLUTION & LOGIC:

Let's trace the recursive execution chain:

`f(4) = 4 + f(3)`

`f(3) = 3 + f(2)`

`f(2) = 2 + f(1)`

`f(1) = 1` (Base Case)

Unwinding back up: `f(2) = 2 + 1 = 3` $\rightarrow$ `f(3) = 3 + 3 = 6` $\rightarrow$ `f(4) = 4 + 6 = 10` .

Answer: 10

GATE CS 2014

Q27. Trace the console print sequence for `fun(3)` :

```
void fun(int n) {  
    if(n > 0) {  
        fun(n-1);  
        printf("%d", n);  
        fun(n-1);  
    }  
}
```

SOLUTION & LOGIC:

This is tree recursion:

- `fun(1)` outputs: 1
- `fun(2)` runs `fun(1)` , prints 2 , runs `fun(1)` $\rightarrow$ outputs: 1 2 1
- `fun(3)` runs `fun(2)` , prints 3 , runs `fun(2)` $\rightarrow$ outputs: 1 2 1 3 1 2 1

Answer: 1213121

GATE CS 2020

Q28. Define Tail Recursion and its primary optimization benefit.

SOLUTION & LOGIC:

Tail recursion occurs when the recursive call is the final operation performed by a function. Compilers can optimize this by reusing the current stack frame (Tail Call Optimization), reducing space complexity from $O(N)$ to $O(1)$.

ISRO CS 2019

Q29. Calculate the minimum moves required to solve the Tower of Hanoi problem with $n = 4$ disks.

SOLUTION & LOGIC:

The minimal structural moves required follows the relation formula: $2^n - 1$

Evaluating for $n = 4$: $2^4 - 1 = 16 - 1 = 15$.

Answer: 15

NIELIT 2022

Q30. Why does linear recursion (like calculating a factorial) require $O(N)$ auxiliary space?

SOLUTION & LOGIC:

Each nested function call creates a new stack frame that remains allocated until the base case resolves. A depth of N calls requires $O(N)$ stack space.

Answer: Due to the accumulation of execution stack frames.

GATE CS 2011

Q31. What is the significance of the Ackermann function $A(m, n)$?

SOLUTION & LOGIC:

The Ackermann function is a classic example of a non-primitive recursive function that grows extremely fast, making it useful for testing compiler stack limits.

UGC NET 2018

Q32. State the failure condition base case for a recursive Binary Search function.

SOLUTION & LOGIC:

The search space is exhausted when the lower bound exceeds the upper bound, indicating the element is missing.

Answer: `if (low > high) return -1;`

GATE CS 2005

Q33. Determine the output generated by calling `bin(13)` :

```
void bin(int n) {
    if(n > 0) {
        bin(n/2);
        printf("%d", n%2);
    }
}
```

SOLUTION & LOGIC:

Tracing the division steps:

`bin(13)` calls `bin(6)` , prints $13 \% 2 = 1$

`bin(6)` calls `bin(3)` , prints $6 \% 2 = 0$

`bin(3)` calls `bin(1)` , prints $3 \% 2 = 1$

`bin(1)` calls `bin(0)` , prints $1 \% 2 = 1$

Printing occurs during stack unwinding: `1 -> 1 -> 0 -> 1` .

Answer: 1101

GATE CS 2009

Q34. Evaluate the McCarthy 91 function output for `f(95)` defined as: $f(n) = n - 10$ if $n > 100$ else $f(f(n + 11))$.

SOLUTION & LOGIC:

This mathematical function evaluates to 91 for any input parameter condition where $n \leq 100$.

Answer: 91

COMPETITIVE EXAM STANDARD

Q35. Predict the output for `rev("cat")` :

```
void rev(char *s) {  
    if(*s) {  
        rev(s+1);  
        printf("%c", *s);  
    }  
}
```

SOLUTION & LOGIC:

The function calls itself recursively until it hits the null terminator `\0`. Characters are printed during stack unwinding, reversing the string.

Answer: tac

ISRO CS 2016

Q36. Compute `gcd(48, 18)` using the Euclidean algorithm: `int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }` .

SOLUTION & LOGIC:

Execution steps:

`gcd(48, 18)` $\rightarrow$ `gcd(18, 48%18 = 12)`

`gcd(18, 12)` $\rightarrow$ `gcd(12, 18%12 = 6)`

`gcd(12, 6)` $\rightarrow$ `gcd(6, 12%6 = 0)`

Since `b == 0` , the function returns `a = 6` .

Answer: 6

GATE CS 2015

Q37. What is the time complexity of a recursive function that splits the exponent in half at each step to compute a^b ?

SOLUTION & LOGIC:

Halving the exponent value at each step using a divide-and-conquer strategy yields a logarithmic time complexity.

Answer: $O(\log b)$

UGC NET 2023

Q38. What type of recursion occurs when Function A calls Function B, and Function B calls Function A?

SOLUTION & LOGIC:

When functions call each other in a cyclic dependency rather than invoking themselves directly, it is known as **Indirect or Mutual Recursion**.

Part 4: Mixed Questions

GATE CS 2019

Q39. Write a recursive strategy to find the maximum value in an integer array.

SOLUTION & LOGIC:

Base case: An array with a single element returns that element: `if(n == 1) return arr[0];`

Recursive step: Compare the last element with the maximum of the remaining array: `return max(arr[n-1], findMax(arr, n-1));`

GATE CS 2017

Q40. In a recursive Binary Search, what expression fills the blank to search the right half of the array: `if (key > arr[mid]) return _____; ?`

SOLUTION & LOGIC:

To search the right half, update the lower bound to `mid + 1` while keeping the upper bound unchanged.

Answer: `binarySearch(arr, mid + 1, high, key)`

GATE CS 2015

Q41. How can you print an array in reverse using recursion without utilizing extra loops or pointer structures?

SOLUTION & LOGIC:

Advance the index recursively until the end of the array is reached, then print the elements during stack unwinding:

```
void printRev(int arr[], int i, int n) {  
    if(i >= n) return;  
    printRev(arr, i+1, n);  
    printf("%d ", arr[i]);  
}
```

GATE CS 2007

Q42. Which fundamental Graph traversal policy mirrors grid-based recursive paths like Flood Fill?

SOLUTION & LOGIC:

The flood-fill algorithm explores a cell and all its neighbors recursively before backtracking, which matches the behavior of **Depth-First Search (DFS)**.

ISRO CS 2017

Q43. What happens when a double pointer `char **str` is passed to a function to modify an array of strings?

SOLUTION & LOGIC:

Passing `char **str` provides the memory address of a string pointer, allowing the function to modify the original string pointer in the array.

GATE CS 2022

Q44. State the base cases for checking if an array is a palindrome using recursive indices `s` (start) and `e` (end).

SOLUTION & LOGIC:

- Successful match: `if(s >= e) return 1;` (All elements verified).
- Mismatch found: `if(arr[s] != arr[e]) return 0;` (Not a palindrome).

NIELIT 2020

Q45. Write the recurrence relation formula for a function that sums all elements in an array.

SOLUTION & LOGIC:

The total sum equals the last element added to the sum of the preceding $n-1$ elements.

Formula: `Sum(A, n) = A[n-1] + Sum(A, n-1)` with base case `Sum(A, 0) = 0`.

UGC NET 2021

Q46. Analyze the worst-case space complexity of a recursive linear search algorithm.

SOLUTION & LOGIC:

If the element is at the end of the array or missing entirely, the function makes N recursive calls, leading to a space complexity of $O(N)$ due to stack frame accumulation.

Answer: $O(N)$

GATE CS 2013

Q47. What happens if a recursive matrix transpose function swaps every element `A[i][j]` with `A[j][i]` across all indices?

SOLUTION & LOGIC:

Swapping every element across all indices processes each pair twice, returning the matrix to its original configuration. To perform a proper transpose, you must swap elements only across the upper or lower triangular region.

GATE CS 2018

Q48. Explain the two recursive choices explored at each step in the Subset Sum problem.

SOLUTION & LOGIC:

For each element, the algorithm checks two branches:

1. Include the element: Subtract its value from the target sum.
2. Exclude the element: Keep the target sum unchanged.

This creates a binary decision tree with a time complexity of $O(2^n)$.

COMPETITIVE EXAM STANDARD

Q49. What is the maximum recursion tree depth when performing a Merge Sort on an array of size N ?

SOLUTION & LOGIC:

Since Merge Sort divides the array in half at each step, the maximum depth of the recursion tree is logarithmic.

Answer: $\log_2 N$

GATE CS 2023

Q50. If a function contains a `static int *p = arr;` statement inside a recursive loop, how does `p` behave across calls?

SOLUTION & LOGIC:

Because `p` is `static`, it is initialized only once. Any modification to the pointer (like `p++`) persists across subsequent recursive calls, rather than resetting to the base address.